

OSSP (Operating Systems and Systems Programming)

Skill Programs – Week 5, Week 6 and Week 7

WEEK - 5

Task 1: Single Quotes

Objective: To apply single quotes, preserve literal content, ignore variable expansion, store quoted strings, validate parsing results, and test edge cases.

Program: task1_single_quotes.c

Code:

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[200];

    printf("Enter text inside single quotes: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    printf("\nOriginal input: %s\n", input);
    printf("Parsed output : %s\n", input);

    printf("\nSingle quotes preserve the content literally.\n");
    printf("Variable expansion and special-character interpretation are ignored.\n");

    return 0;
}
```

OUTPUT :

```
parin@Parinita: ~/OSSP/skill/ x + v
gcc --versionparin@Parinita:~$ gcc --version^C
parin@Parinita:~$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

parin@Parinita:~$ mkdir -p ~/OSSP/skill/week5
cd ~/OSSP/skill/week5
parin@Parinita:~/OSSP/skill/week5$ cd ~/OSSP/skill/week5
nano task1_single_quotes.c
parin@Parinita:~/OSSP/skill/week5$ gcc task1_single_quotes.c -o task1
parin@Parinita:~/OSSP/skill/week5$ ./task1
Enter text inside single quotes: 'Hello $USER'

Original input: 'Hello $USER'
Parsed output : 'Hello $USER'

Single quotes preserve the content literally.
Variable expansion and special-character interpretation are ignored.
parin@Parinita:~/OSSP/skill/week5$ ./task1
Enter text inside single quotes: 'Hello * $HOME 123'

Original input: 'Hello * $HOME 123'
Parsed output : 'Hello * $HOME 123'

Single quotes preserve the content literally.
Variable expansion and special-character interpretation are ignored.
parin@Parinita:~/OSSP/skill/week5$
```

Task 2: Double Quotes

Objective: To apply double quotes, preserve spaces, allow variable expansion, parse nested tokens, validate outputs, and test quoted commands.

Program: task2_double_quotes.c

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char name[100];
    char input[200];

    printf("Enter your name: ");
    fgets(name, sizeof(name), stdin);
```

```

name[strcspn(name, "\n")] = '\0';

setenv("NAME", name, 1);

printf("Enter a double-quoted string containing spaces and $NAME: ");
fgets(input, sizeof(input), stdin);
input[strcspn(input, "\n")] = '\0';

printf("\nOriginal input : \"%s\"\n", input);

char *pos = strstr(input, "$NAME");

if (pos != NULL) {
    char result[200];
    size_t prefix_length = pos - input;

    snprintf(result, sizeof(result),
             "%. *s%s%s",
             (int)prefix_length,
             input,
             getenv("NAME"),
             pos + 5);

    printf("Expanded output: \"%s\"\n", result);
} else {
    printf("Expanded output: \"%s\"\n", input);
}

printf("\nDouble quotes preserve spaces while allowing variable expansion.\n");

return 0;
}

```

OUTPUT :

```

parin@Parinita:~/OSSP/skill/week5$ nano task2_double_quotes.c
parin@Parinita:~/OSSP/skill/week5$ gcc task2_double_quotes.c -o task2
parin@Parinita:~/OSSP/skill/week5$ ./task2
Enter your name: Parin
Enter a double-quoted string containing spaces and $NAME: Hello $NAME, welcome to OSSP

Original input : "Hello $NAME, welcome to OSSP"
Expanded output: "Hello Parin, welcome to OSSP"

Double quotes preserve spaces while allowing variable expansion.
parin@Parinita:~/OSSP/skill/week5$ ./task2
Enter your name: Linux
Enter a double-quoted string containing spaces and $NAME: OSSP $NAME practical program

Original input : "OSSP $NAME practical program"
Expanded output: "OSSP Linux practical program"

Double quotes preserve spaces while allowing variable expansion.

```

WEEK 6

Task 1: Escape Sequences

Objective: To create process escape sequences, handle escaped spaces, escape special symbols, preserve characters, validate parser output, and test complex inputs.

Program: task1_escape_sequences.c

Code:

```

#include <stdio.h>
#include <string.h>

int main() {
    char input[200];
    char output[200];
    int j = 0;

    printf("Enter text with escape characters: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '\\' && input[i + 1] != '\0') {
            i++;

```

```

        output[j++] = input[i];
    } else {
        output[j++] = input[i];
    }
}

output[j] = '\0';

printf("\nOriginal input: %s\n", input);
printf("Parsed output : %s\n", output);

printf("\nEscape characters are processed and the escaped characters are preserved.\n");

return 0;
}

```

OUTPUT:

```

parin@Parinita:~/OSSP/skill/week6$ nano task1_escape_sequences.c
parin@Parinita:~/OSSP/skill/week6$ gcc task1_escape_sequences.c -o task1
parin@Parinita:~/OSSP/skill/week6$ ./task1
Enter text with escape characters: Hello\ World

Original input: Hello\ World
Parsed output : Hello World

Escape characters are processed and the escaped characters are preserved.
parin@Parinita:~/OSSP/skill/week6$ ./task1
Enter text with escape characters: Hello\ \$USER\ \*Test

Original input: Hello\ \$USER\ \*Test
Parsed output : Hello $USER *Test

Escape characters are processed and the escaped characters are preserved.

```

Task 2: Child Processes and Program Execution

Objective: To create child processes, execute programs, handle execution errors, pass arguments, manage the parent process, and test command launching.

Program: task2_child_process.c

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```

#include <unistd.h>
#include <sys/wait.h>

#define MAX_ARGS 20

int main() {
    char input[200];
    char *args[MAX_ARGS];
    int count = 0;

    printf("Enter a command with arguments: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    char *token = strtok(input, " ");

    while (token != NULL && count < MAX_ARGS - 1) {
        args[count++] = token;
        token = strtok(NULL, " ");
    }

    args[count] = NULL;

    if (count == 0) {
        printf("No command entered.\n");
        return 1;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child process created. PID: %d\n", getpid());
        printf("Executing command: %s\n", args[0]);

        execvp(args[0], args);

        perror("Execution failed");
        exit(1);
    }
}

```

```

printf("Parent process. PID: %d\n", getpid());
printf("Waiting for child process...\n");

int status;
waitpid(pid, &status, 0);

if (WIFEXITED(status)) {
    printf("Child process finished with status: %d\n",
        WEXITSTATUS(status));
}

return 0;
}

```

OUTPUT:

```

parin@Parinita:~/OSSP/skill/week6$ nano task2_child_process.c
parin@Parinita:~/OSSP/skill/week6$ gcc task2_child_process.c -o task2
parin@Parinita:~/OSSP/skill/week6$ ./task2
Enter a command with arguments: ls -l
Parent process. PID: 831
Waiting for child process...
Child process created. PID: 832
Executing command: ls
total 44
-rwxr-xr-x 1 parin parin 16192 Sep 21 03:43 task1
-rw-r--r-- 1 parin parin  693 Sep 21 03:43 task1_escape_sequences.c
-rwxr-xr-x 1 parin parin 16488 Sep 21 03:44 task2
-rw-r--r-- 1 parin parin  1229 Sep 21 03:44 task2_child_process.c
Child process finished with status: 0
parin@Parinita:~/OSSP/skill/week6$ ./task2
Enter a command with arguments: doesnotexist
Parent process. PID: 842
Waiting for child process...
Child process created. PID: 843
Executing command: doesnotexist
Execution failed: No such file or directory
Child process finished with status: 1

```

WEEK 7

Task 1: Process Synchronization Using waitpid()

Objective: Process synchronization and child process monitoring using waitpid().

Program: task1_waitpid.c

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child process started.\n");
        printf("Child PID: %d\n", getpid());

        printf("Child is working...\n");
        sleep(3);

        printf("Child process completed.\n");

        exit(42);
    }

    printf("Parent process started.\n");
    printf("Parent PID: %d\n", getpid());
    printf("Waiting for child using waitpid()...\n");

    int status;

    pid_t result = waitpid(pid, &status, 0);

    if (result == -1) {
        perror("waitpid failed");
        return 1;
    }

    if (WIFEXITED(status)) {
        printf("Child terminated normally.\n");
        printf("Child exit status: %d\n", WEXITSTATUS(status));
    }
}
```

```

    }

    printf("Parent process completed.\n");

    return 0;
}

```

OUTPUT:

```

parin@Parinita:~/OSSP/skill/week7$ nano task1_waitpid.c
parin@Parinita:~/OSSP/skill/week7$ gcc task1_waitpid.c -o task1
parin@Parinita:~/OSSP/skill/week7$ ./task1
Parent process started.
Child process started.
Parent PID: 898
Child PID: 899
Child is working...
Waiting for child using waitpid()...
Child process completed.
Child terminated normally.
Child exit status: 42
Parent process completed.

```

Task 2: PATH Variable and Executable Resolution

Objective: To retrieve PATH variable, parse search directories, locate executables, verify permissions, handle missing commands, and test command resolution.

Program: task2_path_resolution.c

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

int main() {
    char *path = getenv("PATH");

    if (path == NULL) {
        printf("PATH variable is not set.\n");
        return 1;
    }

    char command[100];

```

```

printf("Enter executable name: ");
fgets(command, sizeof(command), stdin);
command[strcspn(command, "\n")] = '\0';

char path_copy[8192];

strncpy(path_copy, path, sizeof(path_copy) - 1);
path_copy[sizeof(path_copy) - 1] = '\0';

char *directory = strtok(path_copy, ":");

while (directory != NULL) {
    char full_path[PATH_MAX];

    snprintf(full_path, sizeof(full_path),
             "%s/%s", directory, command);

    if (access(full_path, X_OK) == 0) {
        printf("\nExecutable found: %s\n", full_path);
        printf("Permission check: executable\n");
        return 0;
    }

    directory = strtok(NULL, ":");
}

printf("\nCommand '%s' was not found in PATH or is not executable.\n",
       command);

return 1;
}

```

OUTPUT:

```

parin@Parinita:~/OSSP/skill/week7$ nano task2_path_resolution.c
parin@Parinita:~/OSSP/skill/week7$ gcc task2_path_resolution.c -o task2
parin@Parinita:~/OSSP/skill/week7$ ./task2
Enter executable name: ls

Executable found: /usr/bin/ls
Permission check: executable
parin@Parinita:~/OSSP/skill/week7$ ./task2
Enter executable name: doesnotexist

Command 'doesnotexist' was not found in PATH or is not executable.

```